



Université Ibn Zohr — Ecole Nationale Supérieure de
L'intelligence Artificielle et Sciences des Données
Filière SITCN

PROJET SIBD 2025–2026

Barrage Flow Manager
Version Intelligence Artificielle

Rapport Blue Team

Défense de Sécurité

Analyse OWASP Top 10 · Corrections · Tests de Pénétration

Equipe Blue Team — Pole Securite

Defenseuse

IGHRANE Imane

Defenseur

HRIMICH Reda

Encadrant

Pr. S. EL-ATEIF

Branche GitHub

security/blue-team-defense

2025-2026

Table des matières

1	Introduction et Contexte	3
1.1	Présentation du Projet	3
1.2	Architecture Technique	3
1.3	Rôle de la Blue Team	3
1.4	Méthodologie d'Audit	3
1.5	Outils Utilisés par la Blue Team	4
2	Analyse OWASP Top 10	5
3	Corrections Appliquées	6
3.1	Fix 1 : Injection SQL (A03)	6
3.1.1	Vulnérabilité Détectée	6
3.1.2	Vulnérabilité Identifiée	6
3.1.3	Code Avant Correction	7
3.1.4	Solution Implémentée	7
3.1.5	Code Après Correction	7
3.1.6	Fichiers vérifiés — Aucune faille	7
3.2	Fix 2 : Validation des Données Pydantic (A03/A04)	8
3.2.1	Vulnérabilité Détectée	8
3.2.2	Vulnérabilité Identifiée	8
3.2.3	Code Avant Correction	9
3.2.4	Solution Implémentée	9
3.2.5	Code Après Correction	10
3.3	Fix 3 : Protection Anti Brute-Force / Rate Limiting (A07)	10
3.3.1	Vulnérabilité Détectée	10
3.3.2	Vulnérabilité Identifiée	11
3.3.3	Code Avant Correction	11
3.3.4	Solution Implémentée	11
3.3.5	Code Après Correction	12
3.4	Fix 4 : CORS & Méthodes HTTP (A05)	12
3.4.1	Vulnérabilité Détectée	12
3.4.2	Vulnérabilité Identifiée	13
3.4.3	Code Avant / Après	13
3.5	Fix 5 : RBAC — Contrôle d'Accès par Rôle (A01)	13
3.5.1	Vulnérabilité Détectée	14
3.5.2	Vulnérabilité Identifiée	14
3.5.3	Code Avant Correction	15
3.5.4	Solution Implémentée	15

3.5.5	Code Après Correction	16
3.6	Fix 6 : Hashage bcrypt & Sécurité MySQL (A02)	16
3.6.1	Vulnérabilité Détectée	17
3.6.2	Vulnérabilité Identifiée	17
3.6.3	Code Avant Correction	17
3.6.4	Solution Implémentée	18
3.6.5	Code Après Correction	18
3.7	Fix 7 : Port Docker MySQL Exposé (A04)	18
3.7.1	Vulnérabilité Détectée	19
3.7.2	Code Avant / Après	19
3.8	Fix 8 : Vérification .gitignore (A02)	19
3.8.1	Vulnérabilité Vérifiée	19
3.8.2	Résultat	20
4	Tests de Vérification — Résultats	21
4.1	T-SQL-01 — Injection SQL	21
4.2	T-PYD-01 & T-PYD-02 — Validation Pydantic	22
4.3	T-PYD-03 — Mot de Passe Trop Court	24
4.4	T-CORS-01 — CORS & Origine Malveillante	25
4.5	T-RATE-01 — Rate Limiting	27
4.6	T-RBAC — Contrôle d'Accès	28
4.7	T-HASH-01 — Hashage des Mots de Passe	31
4.8	T-MYSQL — Permissions MySQL & Port	32
5	Bilan Final	35
5.1	Tableau Récapitulatif des Tests	35
5.2	Statistiques	35
5.3	Conclusion et Remerciements	36

1. Introduction et Contexte

1.1 Présentation du Projet

Le **Barrage-Flow-Manager** est un système de gestion hydrique stratégique conçu pour le barrage **Youssef Ibn Tachfine** dans la région Souss-Massa. Ce système permet l'arbitrage sécurisé entre l'irrigation agricole et les réserves vitales d'eau potable (AEP).

1.2 Architecture Technique

Couche	Technologie	Notes
Frontend	React + Vite + TypeScript	Dashboard Recharts, Leaflet
Backend API	FastAPI (Python 3.11+)	REST API · JWT · RBAC
Base de données	MySQL 8.0	Triggers · Procédures stockées
Conteneurisation	Docker + Docker Compose	Environnement unifié
Versioning	Git + GitHub	Workflow PR + Approval

TABLE 1.1 – Stack technique du projet Barrage Flow Manager

1.3 Rôle de la Blue Team

Mission de la Blue Team

- Durcir la sécurité du système Barrage-Flow-Manager
- Vérifier et renforcer le RBAC (seul le Directeur peut forcer un lâcher d'eau)
- Protéger les données critiques (niveaux de réserve, stocks AEP)
- Préparer le rapport Post-Fix livrable

1.4 Méthodologie d'Audit

La méthodologie s'est déroulée en 4 phases principales :

1. **Exploration de l'architecture** : Lecture complète de tous les fichiers backend, frontend, base de données et configuration Docker.
2. **Analyse statique du code** : Vérification ligne par ligne de chaque route API, middleware, schéma et configuration.

3. **Identification des vulnérabilités** : Comparaison avec le référentiel **OWASP Top 10 2021**.
4. **Implémentation des correctifs** : Modification du code avec tests de vérification et rapport détaillé.

1.5 Outils Utilisés par la Blue Team

#	Outil	Usage dans le Projet
1	 VS Code	Éditeur principal pour analyser et modifier le code backend Python
2	 Postman	Test des endpoints API (login, RBAC, rate limiting, injection)
3	 GitHub.com	Gestion des Pull Requests et commentaires de révision
4	 GitHub Desktop	Interface graphique pour les commits et push
5	 XAMPP	Serveur MySQL local · Vérification des privilèges · php-MyAdmin
6	 CMD	Tests connexion MySQL, vérification port 3306, commandes Docker

TABLE 1.2 – Outils utilisés par la Blue Team pour l’audit et les correctifs

2. Analyse OWASP Top 10

#	Catégorie	Statut	Résultat
A01	Broken Access Control	RBAC manquant sur routes sensibles	Corrigé
A02	Cryptographic Failures	Mots de passe en clair · SE-CRET_KEY faible	Corrigé
A03	Injection	f-string SQL dans releases.py	Corrigé
A04	Insecure Design	Port 3306 exposé · root MySQL	Corrigé
A05	Security Misconfiguration	CORS permissif · .env non protégé	Corrigé
A06	Vulnerable Components	Dépendances non vérifiées	Identifié
A07	Authentication Failures	Pas de rate limiting sur /login	Corrigé
A08	Software Integrity	N/A	N/A
A09	Logging & Monitoring	Aucun log sur tentatives échouées	Identifié
A10	SSRF	N/A	N/A

TABLE 2.1 – Couverture OWASP Top 10 — Projet Barrage Flow Manager

3. Corrections Appliquées

3.1 Fix 1 : Injection SQL (A03)

3.1.1 Vulnérabilité Détectée

Méthodologie de Détection

Lors de l'analyse du fichier `releases.py`, il a été constaté qu'une f-string Python construisait directement la requête SQL avec la valeur `id_lacher` sans paramétrage.

```
1 # Code original observe dans releases.py :
2 db.execute(text(f"CALL sp_repartir_eau({id_lacher})"))
3 # Aucun parametre nomme ! Injection possible.
```

Analyse OWASP : Correspond à **A03 — Injection**. Un attaquant peut injecter du SQL via `id_lacher`.

3.1.2 Vulnérabilité Identifiée

Vulnérabilité : Injection SQL via f-string

- **Type** : OWASP A03 — Injection
- **Sévérité** : **ÉLEVÉ**
- **Fichier** : `backend/app/routes/releases.py`
- **Impact** : Suppression de tables, création de comptes

Impact & Risque

- `id_lacher = "1"); DROP TABLE Lacher_Eau; --` → suppression complète de l'historique
- `id_lacher = "1"); INSERT INTO Utilisateur VALUES(...); --` → création d'un compte Directeur non autorisé

3.1.3 Code Avant Correction

✗ AVANT — Code Vulnérable

```

1 # backend/app/routes/releases.py -- AVANT
2 # DANGEREUX : f-string directement dans la requete SQL
3 db.execute(text(f"CALL␣sp_repartir_eau({id_lacher})"))
4 #
5 #          Valeur non sanitisee          injection possible !

```

3.1.4 Solution Implémentée

✓ Solution Implémentée

Remplacement de la f-string par un paramètre nommé SQLAlchemy, isolant complètement la valeur de la requête SQL.

3.1.5 Code Après Correction

✓ APRÈS — Code Corrigé

```

1 # backend/app/routes/releases.py -- APRES
2 # SECURISE : parametre nomme separe de la requete
3 db.execute(
4     text("CALL␣sp_repartir_eau(:id)"),
5     {"id": id_lacher}    # <- valeur isolee
6 )
7 # Injection impossible :
8 # id_lacher = "1"); DROP TABLE Lacher_Eau; --"
9 # -> Rejetee : 422 "Input should be a valid integer"

```

3.1.6 Fichiers vérifiés — Aucune faille

Fichier	Méthode utilisée	Statut
auth.py	SQLAlchemy ORM	Sûr
alerts.py	Paramètres nommés text()	Sûr
dashboard.py	Procédures stockées	Sûr
lacher_eau.py	Paramètres nommés	Sûr
repartition.py	Paramètres nommés	Sûr
users.py	SQLAlchemy ORM	Sûr
releases.py	f-string (corrigé)	Corrigé

TABLE 3.1 – Vérification complète des routes — Injection SQL

3.2 Fix 2 : Validation des Données Pydantic (A03/A04)

3.2.1 Vulnérabilité Détectée

Q Méthodologie de Détection

Lors de l’analyse des fichiers du dossier backend/app/schemas/, chaque schéma Pydantic a été examiné :

```
1 # Code original dans user.py :
2 class UserCreate(BaseModel):
3     password: str # Aucune contrainte de longueur !
4
5 # Dans lacher_eau.py :
6 class LacherEauCreate(BaseModel):
7     id_user: int # Accepte les valeurs negatives !
8     id_barrage: int # Accepte les valeurs negatives !
```

Analyse OWASP : Correspond à **A03/A04 — Insecure Design**. Des valeurs invalides peuvent créer des incohérences en base.

3.2.2 Vulnérabilité Identifiée

! Vulnérabilité : Validation Pydantic insuffisante

- **Type** : OWASP A03/A04
- **Fichiers** : user.py, dashboard.py, alerte.py, demande_irrigation.py, cooperative.py, lacher_eau.py

Fichier	Faille	Risque
user.py	password sans min_length	ÉLEVÉ
dashboard.py	List[dict] non typé	MOYEN
alerte.py	id_barrage sans gt=0	MOYEN
demande_irrigation.py	id_coop sans gt=0	MOYEN
cooperative.py	historique négatif	FAIBLE
lacher_eau.py	id_user/id_barrage sans gt=0	MOYEN

3.2.3 Code Avant Correction

✗ AVANT — Code Vulnérable

```
1 # schemas/user.py -- AVANT
2 class UserCreate(BaseModel):
3     nom: str
4     email: EmailStr
5     password: str          # <- accepte "a", "1", etc.
6     role: UserRole
7
8 # schemas/dashboard.py -- AVANT
9 class DashboardResponse(BaseModel):
10     alertes_urgentes: List[dict] # <- n'importe quoi accepte
```

3.2.4 Solution Implémentée

✓ Solution Implémentée

Ajout de contraintes `Field()` sur tous les champs critiques : `min_length=8` pour les mots de passe, `gt=0` pour les IDs et volumes, `ge=0` pour les valeurs positives ou nulles. Création d'un schéma `AlerteUrgente` typé pour remplacer `List[dict]`.

3.2.5 Code Après Correction

✓ APRÈS — Code Corrigé

```

1  # schemas/user.py -- APRES
2  class UserCreate(BaseModel):
3      nom: str = Field(..., min_length=2, max_length=100)
4      email: EmailStr
5      password: str = Field(..., min_length=8) # <- securise
6      role: UserRole
7
8  # schemas/dashboard.py -- APRES
9  class AlerteUrgente(BaseModel): # <- nouveau schema strict
10     id_alerte: int
11     type: str
12     message: str
13     id_barrage: int
14
15  class DashboardResponse(BaseModel):
16     alertes_urgentes: List[AlerteUrgente] # <- valide
17
18  # schemas/alerte.py -- APRES
19  class AlerteCreate(BaseModel):
20     id_barrage: int = Field(..., gt=0) # <- uniquement > 0
21
22  # schemas/lacher_eau.py -- APRES
23  class LacherEauCreate(BaseModel):
24     id_user: int = Field(..., gt=0) # <- uniquement > 0
25     id_barrage: int = Field(..., gt=0) # <- uniquement > 0

```

3.3 Fix 3 : Protection Anti Brute-Force / Rate Limiting (A07)

3.3.1 Vulnérabilité Détectée

Q Méthodologie de Détection

Lors de l'analyse du fichier backend/app/routes/auth.py, il a été constaté que l'endpoint /login ne disposait d'aucune protection contre les tentatives répétées.

```

1  # Code original dans auth.py :
2  @router.post("/login")
3  def login(user_data: UserLogin, db: Session = Depends(get_db)):
4      # Aucune limite de tentatives !
5      ...

```

Analyse OWASP : Correspond à **A07 — Authentication Failures**. Un attaquant peut envoyer des milliers de requêtes par seconde.

3.3.2 Vulnérabilité Identifiée

⚠ Vulnérabilité : Absence de Rate Limiting sur le Login

- **Type** : OWASP A07 — Authentication Failures
- **Sévérité** : **ÉLEVÉ**
- **Fichier** : backend/app/routes/auth.py
- **Impact** : Brute-force possible sans limite

💀 Impact & Risque

Un attaquant peut tester 10 000+ mots de passe par minute sur le compte Directeur, compromettant l'accès total au barrage en quelques minutes.

3.3.3 Code Avant Correction

✗ AVANT — Code Vulnérable

```
1 # auth.py -- AVANT
2 @router.post("/login")
3 def login(user_data: UserLogin, db: Session = Depends(get_db)):
4     """Aucune protection contre le brute-force."""
5     user = db.query(User).filter(
6         User.email == user_data.email).first()
7     ...
```

3.3.4 Solution Implémentée

✓ Solution Implémentée

Installation de la bibliothèque `slowapi` et limitation à **5 tentatives de login par minute** par adresse IP. Configuration globale du limiter dans `main.py`.

3.3.5 Code Après Correction

✓ APRÈS — Code Corrigé

```

1  # auth.py -- APRES
2  from slowapi import Limiter
3  from slowapi.util import get_remote_address
4
5  limiter = Limiter(key_func=get_remote_address)
6
7  @router.post("/login")
8  @limiter.limit("5/minute")    # <- max 5 tentatives/min/IP
9  def login(request: Request, user_data: UserLogin,
10          db: Session = Depends(get_db)):
11      """Authentification avec rate limiting."""
12      ...
13
14  # main.py -- APRES (configuration globale)
15  from slowapi import Limiter, _rate_limit_exceeded_handler
16  from slowapi.errors import RateLimitExceeded
17
18  limiter = Limiter(key_func=get_remote_address)
19  app.state.limiter = limiter
20  app.add_exception_handler(
21      RateLimitExceeded, _rate_limit_exceeded_handler
22  )

```

3.4 Fix 4 : CORS & Méthodes HTTP (A05)

3.4.1 Vulnérabilité Détectée

Q Méthodologie de Détection

Lors de la lecture du fichier backend/app/main.py, la configuration CORS suivante a été relevée :

```

1  # Code original observe dans main.py :
2  app.add_middleware(
3      CORSMiddleware,
4      allow_origins=["http://localhost:5173"],
5      allow_methods=["*"],    # TOUTES les methodes !
6      allow_headers=["*"],    # TOUS les headers !
7  )

```

Analyse OWASP : Correspond à **A05 — Security Misconfiguration**.

3.4.2 Vulnérabilité Identifiée

⚠ Vulnérabilité : CORS trop permissif

- **Type** : OWASP A05 — Security Misconfiguration
- **Sévérité** : **MOYEN**
- **Fichier** : backend/app/main.py
- **Impact** : Méthodes HTTP non nécessaires autorisées

3.4.3 Code Avant / Après

✗ AVANT — Code Vulnérable

```
1 # main.py -- AVANT
2 app.add_middleware(
3     CORSMiddleware,
4     allow_origins=["http://localhost:5173"],
5     allow_methods=["*"],      # DANGEREUX
6     allow_headers=["*"],      # DANGEREUX
7 )
```

✓ APRÈS — Code Corrigé

```
1 # main.py -- APRES
2 app.add_middleware(
3     CORSMiddleware,
4     allow_origins=[
5         "http://localhost:5173",
6         "http://127.0.0.1:5173"
7     ],
8     allow_credentials=True,
9     allow_methods=["GET", "POST", "PUT", "DELETE"], # Restreint
10    allow_headers=["Authorization", "Content-Type"],
11    max_age=600,
12 )
```

3.5 Fix 5 : RBAC — Contrôle d'Accès par Rôle (A01)

3.5.1 Vulnérabilité Détectée

🔍 Méthodologie de Détection

Lors de l’analyse des routes dans backend/app/routes/, la présence du middleware RoleChecker a été vérifiée sur chaque endpoint sensible.

```
1 # Code original dans lacher_eau.py :
2 @router.post("/")
3 def create_release(
4     payload: ReleaseRequest,
5     db: Session = Depends(get_db),
6     current_user: dict = Depends(get_current_user),
7     # Pas de verification de role !
8 ):

```

Analyse OWASP : Correspond à **A01 — Broken Access Control**.

3.5.2 Vulnérabilité Identifiée

⚠️ Vulnérabilité : RBAC manquant sur routes sensibles

- **Type** : OWASP A01 — Broken Access Control
- **Sévérité** : **ÉLEVÉ**

Route	Faible	Risque
POST /releases/	Tout utilisateur peut créer	ÉLEVÉ
POST/PUT/DELETE /repartitions/	Sans restriction	ÉLEVÉ
GET /alerts/critical	Visible par tous	MOYEN
dependencies.py	Sans return payload	MOYEN

💀 Impact & Risque

Sans RBAC sur /releases/ : un simple opérateur peut déclencher un lâcher d’eau massif sans autorisation, mettant en danger les réserves vitales AEP de la région Souss-Massa.

3.5.3 Code Avant Correction

✗ AVANT — Code Vulnérable

```
1 # lacher_eau.py -- AVANT
2 @router.post("/")
3 def create_release(
4     payload: ReleaseRequest,
5     db: Session = Depends(get_db),
6     current_user: dict = Depends(get_current_user),
7     # <- pas de verification de role !
8 ):
9
10 # dependencies.py -- AVANT
11 class RoleChecker:
12     def __call__(self, payload: dict = Depends(get_current_user)
13 ):
14     role = payload.get("role")
15     if role not in self.allowed_roles:
16         raise HTTPException(status_code=403,
17                             detail="Access forbidden")
18     # <- rien retourne !
```

3.5.4 Solution Implémentée

✓ Solution Implémentée

Ajout de RoleChecker sur toutes les routes sensibles et correction du return payload manquant dans dependencies.py.

3.5.5 Code Après Correction

✓ APRÈS — Code Corrigé

```
1 # lacher_eau.py -- APRES
2 @router.post("/")
3 def create_release(
4     payload: ReleaseRequest,
5     db: Session = Depends(get_db),
6     current_user: dict = Depends(
7         RoleChecker(["Directeur", "directeur",
8                     "Gestionnaire", "gestionnaire"])
9     ), # <- role verifie
10 ):
11
12 # alerts.py -- APRES
13 @router.get("/critical")
14 def get_critical_alerts(
15     db: Session = Depends(get_db),
16     current_user: dict = Depends(
17         RoleChecker(["Directeur", "directeur",
18                     "Gestionnaire", "gestionnaire"])
19     ),
20 ):
21
22 # dependencies.py -- APRES
23 class RoleChecker:
24     def __call__(self, payload: dict = Depends(get_current_user)
25     ):
26         role = payload.get("role")
27         if role not in self.allowed_roles:
28             raise HTTPException(status_code=403,
29                                 detail="Access_␣forbidden")
29         return payload # <- donnees utilisateur transmises
```

3.6 Fix 6 : Hashage bcrypt & Sécurité MySQL (A02)

3.6.1 Vulnérabilité Détectée

🔍 Méthodologie de Détection

Lors de la lecture des fichiers `security.py`, `docker-compose.yml` et `02_seed_data.sql`, plusieurs failles critiques ont été constatées :

```
1 # security.py -- Code original observe :
2 SECRET_KEY = os.getenv("SECRET_KEY", "secret")
3 # <- cle triviale par défaut !
4
5 def hash_password(password: str) -> str:
6     return password # <- stockage EN CLAIR !
```

```
1 -- docker-compose.yml -- Code observe :
2 MYSQL_ROOT_PASSWORD=rootpassword -- Faible !
3 -- L'API se connecte en tant que ROOT MySQL !
```

Analyse OWASP : Correspond à **A02 — Cryptographic Failures**.

3.6.2 Vulnérabilité Identifiée

⚠️ Vulnérabilité : Cryptographie insuffisante & MySQL root

- Type : OWASP A02 — Cryptographic Failures
- Sévérité : **CRITIQUE**

Fichier	Faible	Risque
<code>docker-compose.yml</code>	API en root MySQL	CRITIQUE
<code>.env</code>	DATABASE_URL sur root	CRITIQUE
<code>02_seed_data.sql</code>	Mots de passe en clair	CRITIQUE
<code>security.py</code>	SECRET_KEY "secret"	CRITIQUE

💀 Impact & Risque

Root MySQL compromis = DROP de toutes les tables possible. Mots de passe en clair = un dump SQL suffit à tout compromettre. SECRET_KEY triviale = forgerie de tokens JWT Directeur possible.

3.6.3 Code Avant Correction

✖ AVANT — Code Vulnérable

```
1 # security.py -- AVANT
2 SECRET_KEY = os.getenv("SECRET_KEY", "secret") # <- trivial !
3
4 def hash_password(password: str) -> str:
5     return password # <- stockage en clair !
```

3.6.4 Solution Implémentée

✓ Solution Implémentée

Hashage bcrypt via `passlib`, clé secrète 64 caractères avec crash au démarrage si absente. Création de l'utilisateur MySQL dédié `barrage_app` avec droits minimaux uniquement.

3.6.5 Code Après Correction

✓ APRÈS — Code Corrigé

```

1  # security.py -- APRES
2  SECRET_KEY = os.getenv("SECRET_KEY")
3  if not SECRET_KEY:
4      raise ValueError(
5          "SECRET_KEY manquante - arrêt du serveur"
6      )
7
8  pwd_context = CryptContext(
9      schemes=["bcrypt"], deprecated="auto"
10 )
11
12 def hash_password(password: str) -> str:
13     return pwd_context.hash(password) # <- irreversible
14
15 def verify_password(plain: str, hashed: str) -> bool:
16     return pwd_context.verify(plain, hashed)

```

```

1  -- 06_app_user.sql -- NOUVEAU FICHIER
2  CREATE USER IF NOT EXISTS 'barrage_app'@'%'
3      IDENTIFIED BY 'AppSecure2024!';
4
5  GRANT SELECT, INSERT, UPDATE, DELETE, EXECUTE
6      ON barrage_flow_db_AI_Version.*
7      TO 'barrage_app'@'%';
8  -- DROP, CREATE, ALTER, GRANT sont REFUSES
9  FLUSH PRIVILEGES;

```

3.7 Fix 7 : Port Docker MySQL Exposé (A04)

3.7.1 Vulnérabilité Détectée

Q Méthodologie de Détection

Lors de la lecture du fichier `docker-compose.yml`, il a été observé que le port MySQL était exposé publiquement :

```
1 # docker-compose.yml -- Code original :
2 services:
3   database:
4     ports:
5     - "3306:3306" # <- accessible depuis internet !
```

Analyse OWASP : Correspond à **A04 — Insecure Design**.

3.7.2 Code Avant / Après

✗ AVANT — Code Vulnérable

```
1 # docker-compose.yml -- AVANT
2 services:
3   database:
4     image: mysql:8.0
5     ports:
6     - "3306:3306" # DANGEREUX : accessible publiquement
```

✓ APRÈS — Code Corrigé

```
1 # docker-compose.yml -- APRES
2 services:
3   database:
4     image: mysql:8.0
5     # ports: supprime
6     # MySQL accessible uniquement en interne Docker
```

3.8 Fix 8 : Vérification .gitignore (A02)

3.8.1 Vulnérabilité Vérifiée

Q Méthodologie de Détection

Lors de la lecture du fichier `.gitignore` à la racine du projet, il a été vérifié que le fichier `.env` était bien présent dans la liste des fichiers ignorés, afin d'éviter tout envoi accidentel des secrets sur GitHub.

3.8.2 Résultat

Résultat de la vérification

Le fichier `.env` est correctement listé dans `.gitignore`. Les secrets (clés JWT, mots de passe MySQL) ne sont jamais envoyés sur GitHub. Statut : **Déjà conforme — aucune modification nécessaire.**

4. Tests de Vérification — Résultats

4.1 T-SQL-01 — Injection SQL

Test de Vérification

Méthode	PUT
URL	/api/releases/1)%3B%20DROP%20TABLE%20test/execute
Payload	"1); DROP TABLE test; -"
Résultat attendu	422 Unprocessable Entity
Résultat obtenu	422 — "Input should be a valid integer"
Statut	INJECTION BLOQUEE

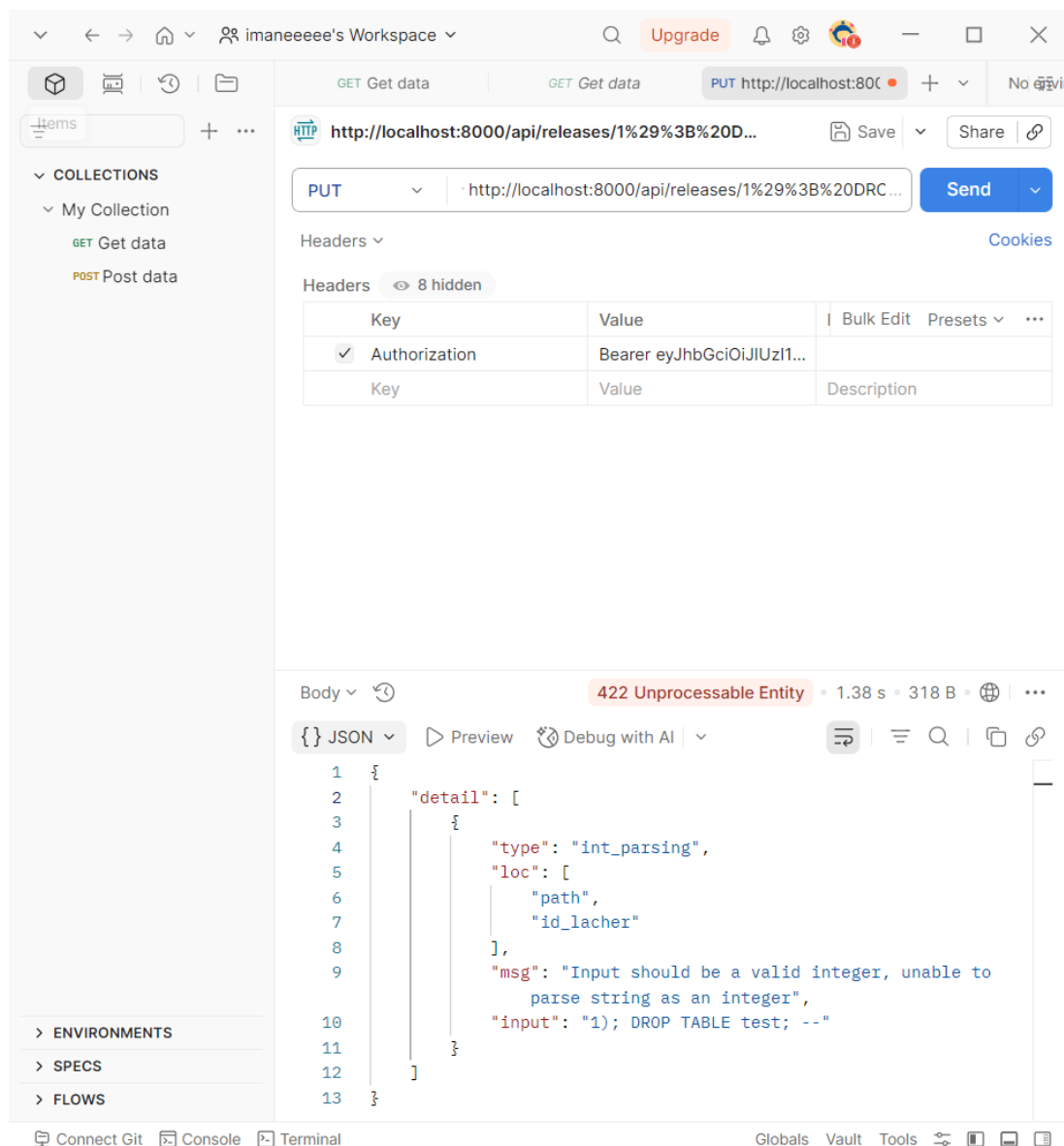


FIGURE 4.1 – T-SQL-01 — Injection SQL rejetée — HTTP 422

4.2 T-PYD-01 & T-PYD-02 — Validation Pydantic

🔧 Test de Vérification

Test 1 : volume_m3 = -100 → attendu 422

Test 2 : id_barrage = -5 → attendu 422

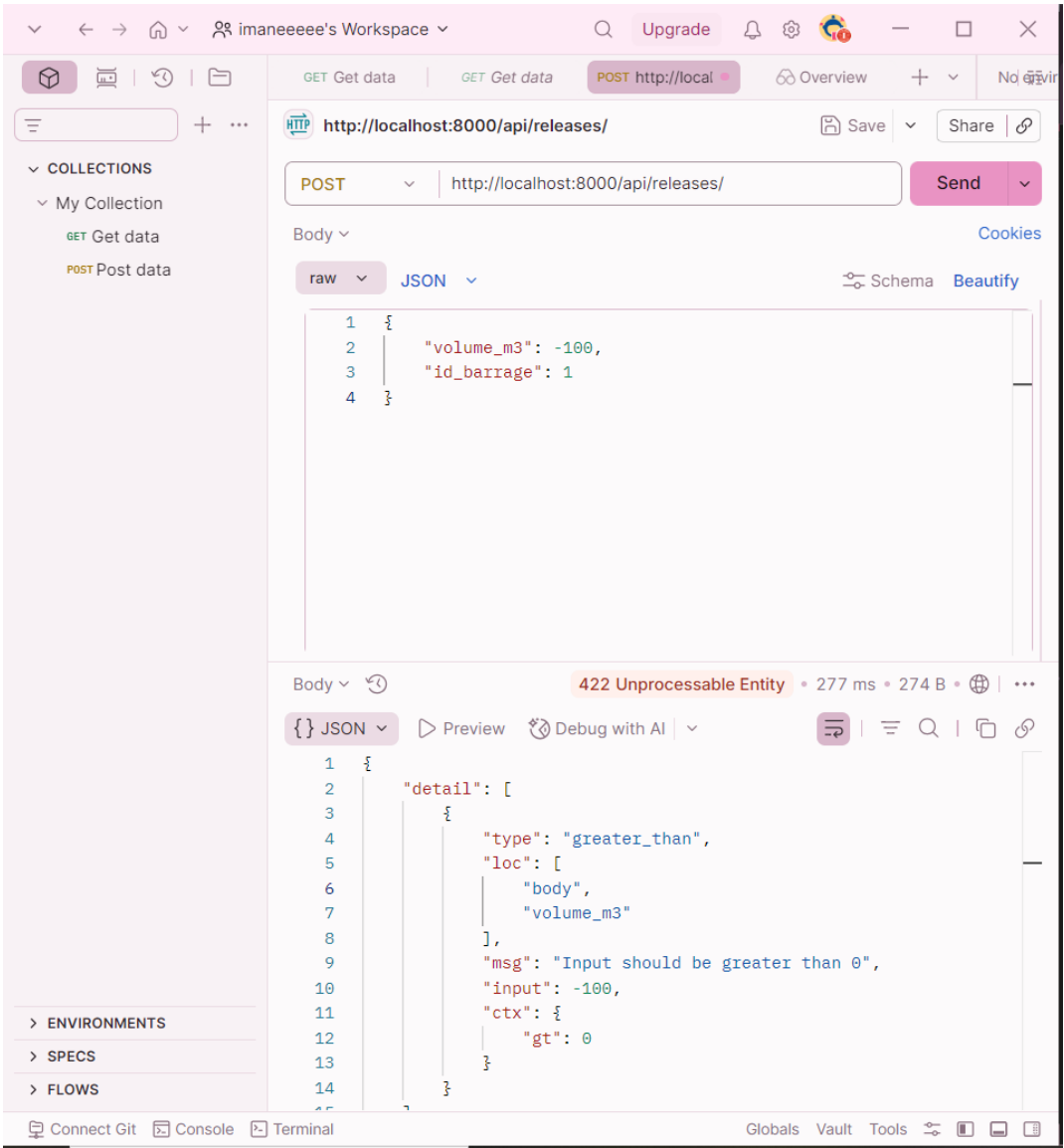
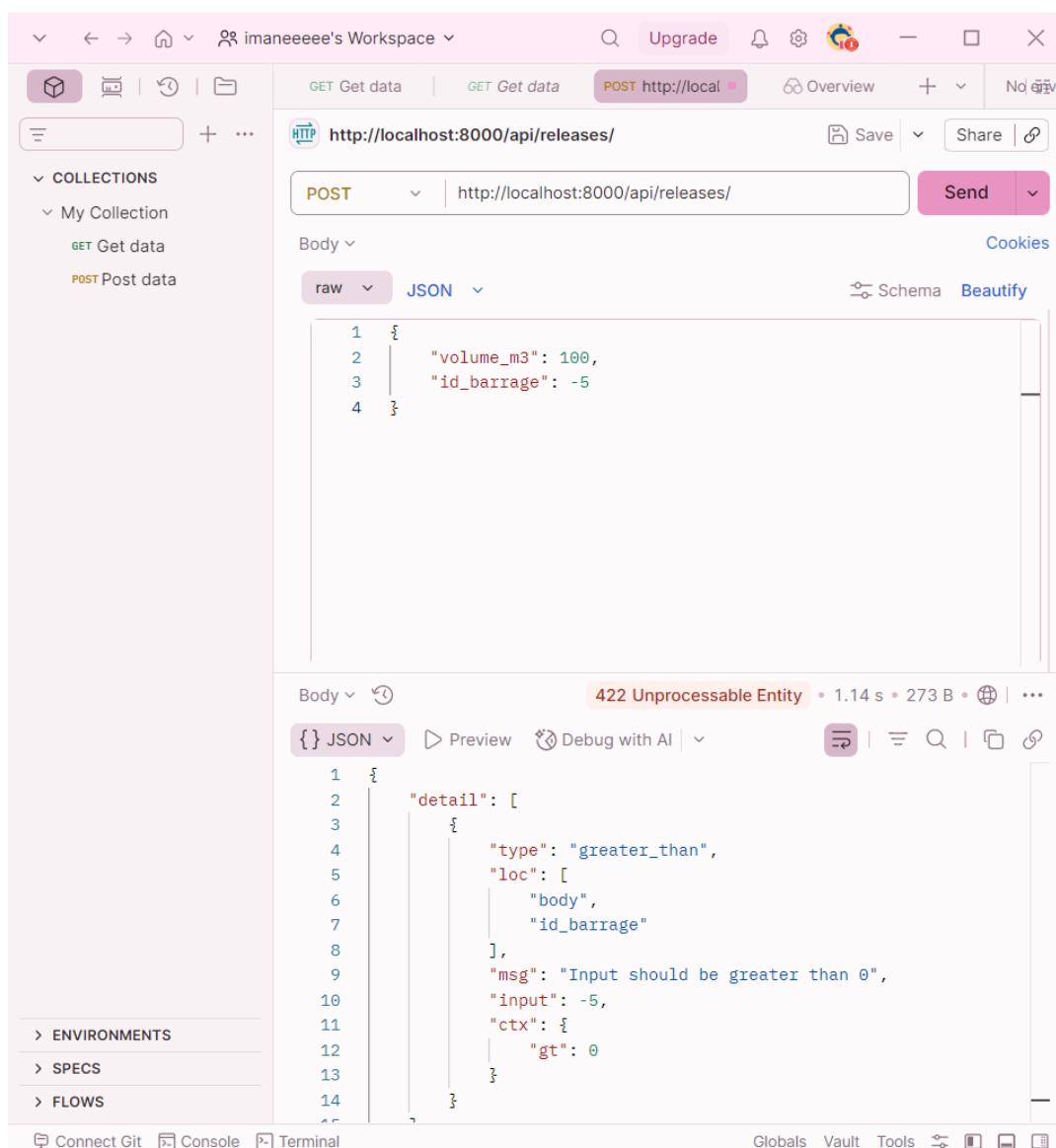


FIGURE 4.2 – T-PYD-01 — volume_m3 = -100 → HTTP 422

FIGURE 4.3 – T-PYD-02 — `id_barrage = -5` → HTTP 422

4.3 T-PYD-03 — Mot de Passe Trop Court

Test de Vérification

Test : `password = "ab"` → attendu 422 Unprocessable Entity

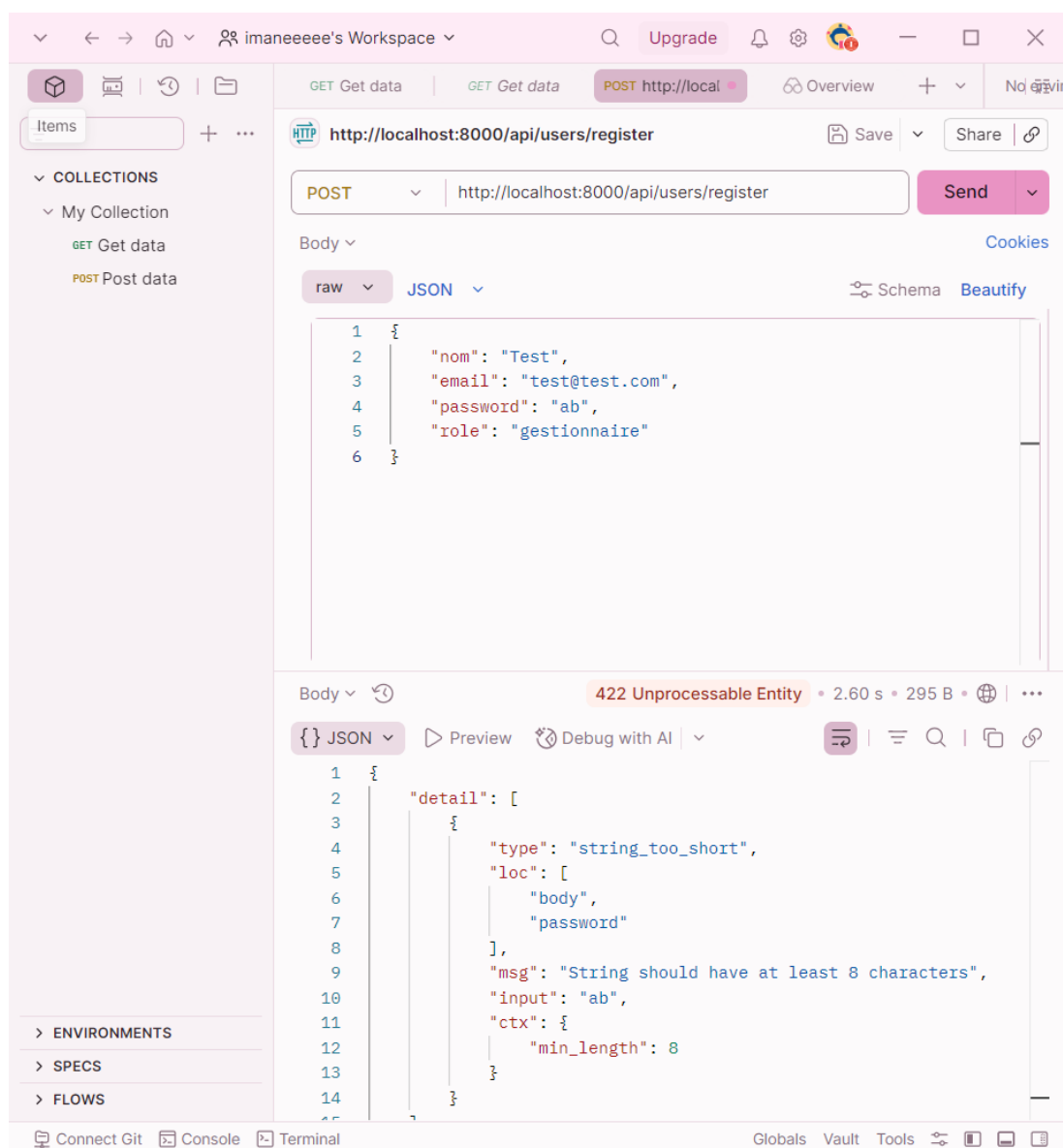


FIGURE 4.4 – T-PYD-03 — password = "ab" → HTTP 422 "String should have at least 8 characters"

4.4 T-CORS-01 — CORS & Origine Malveillante

🔍 Test de Vérification

Test 1 : GET avec Origin : site-malveillant.com → attendu : pas de * dans CORS

Test 2 : OPTIONS → attendu : 405 Method Not Allowed

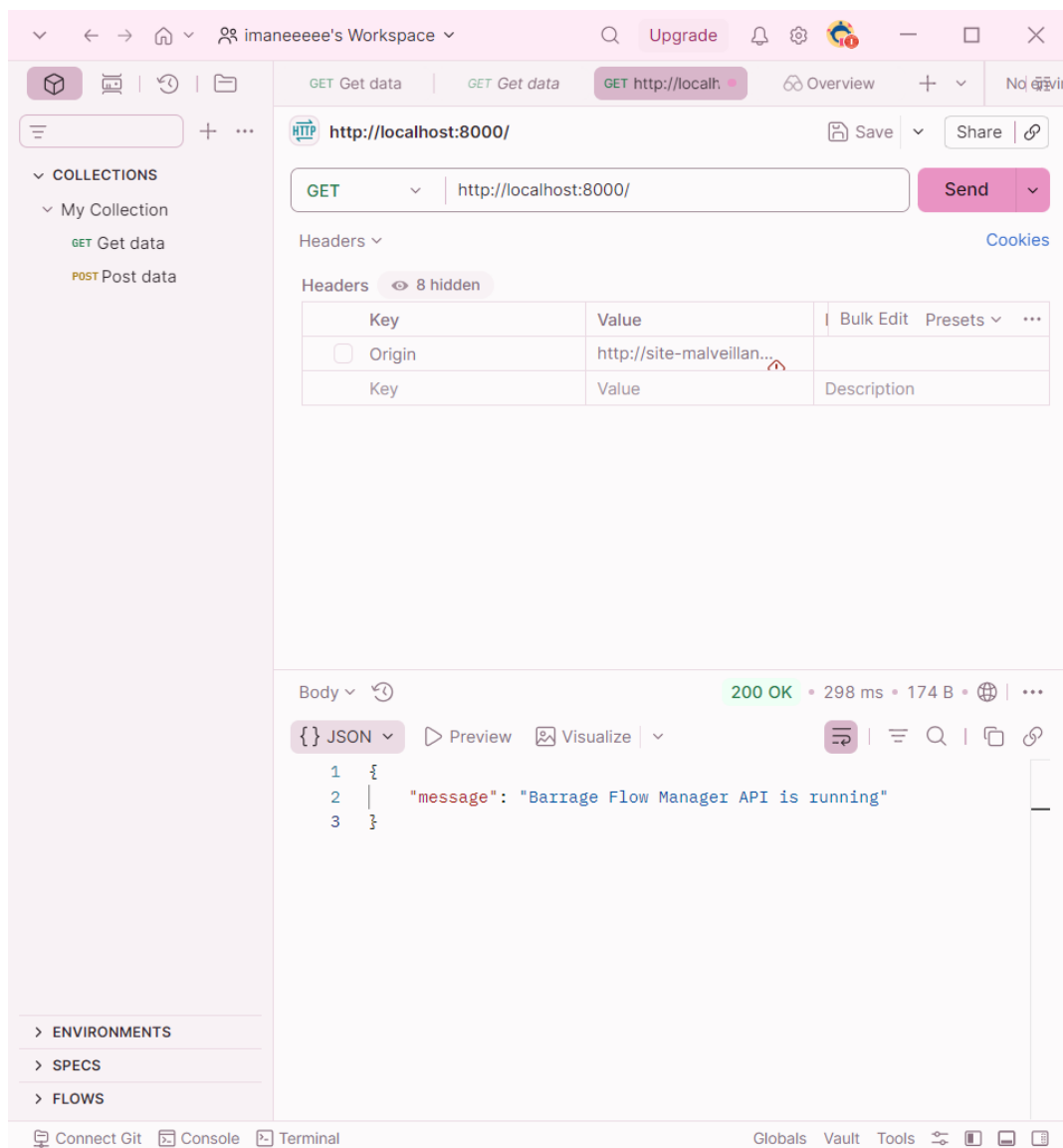


FIGURE 4.5 – T-CORS-01 — GET avec origine malveillante

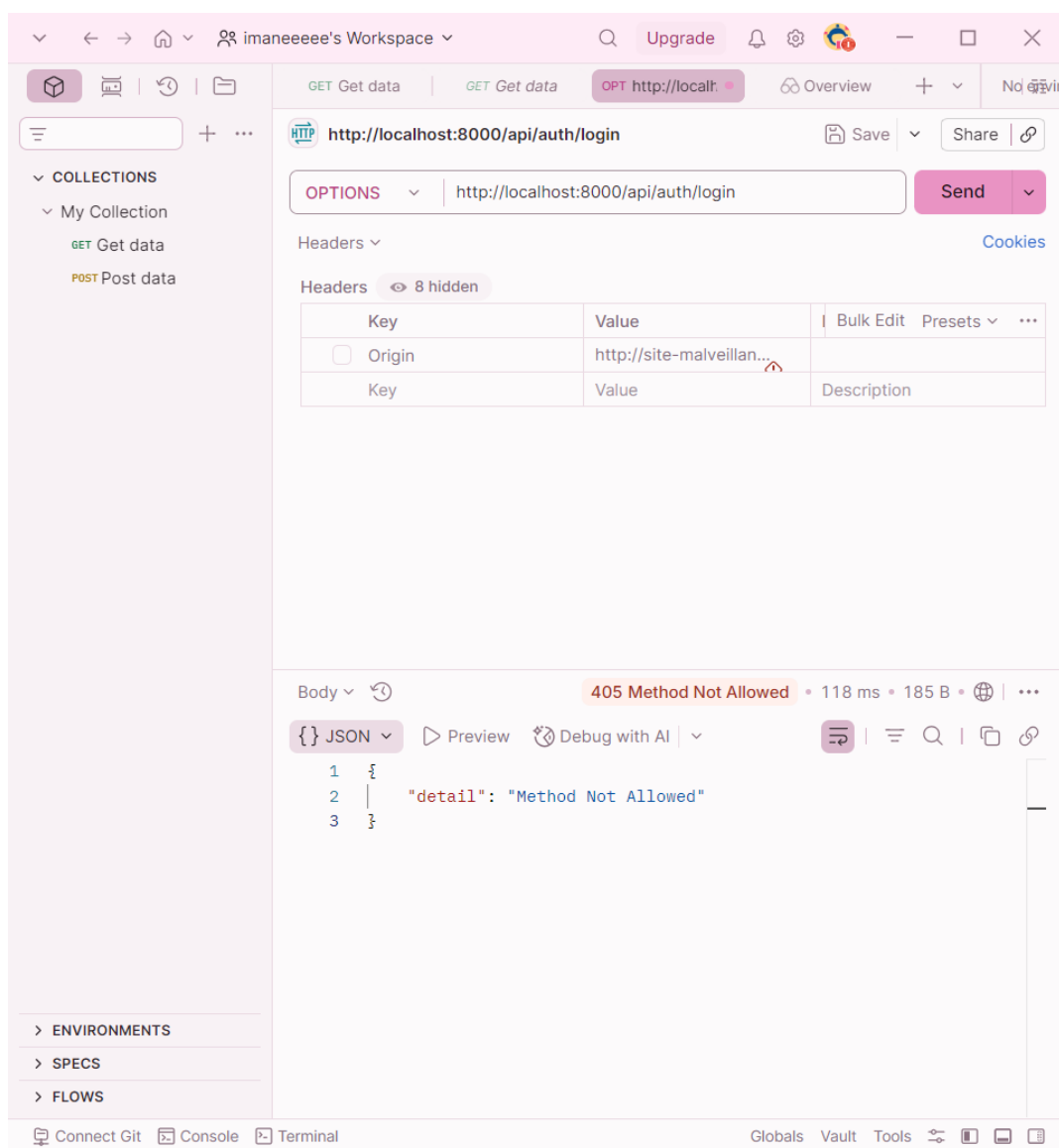


FIGURE 4.6 – T-CORS-02 — OPTIONS → 405 Method Not Allowed

4.5 T-RATE-01 — Rate Limiting

🔧 Test de Vérification

Test : 6 requêtes POST /api/auth/login en moins d'1 minute

Résultat attendu à la 6e : 429 Too Many Requests

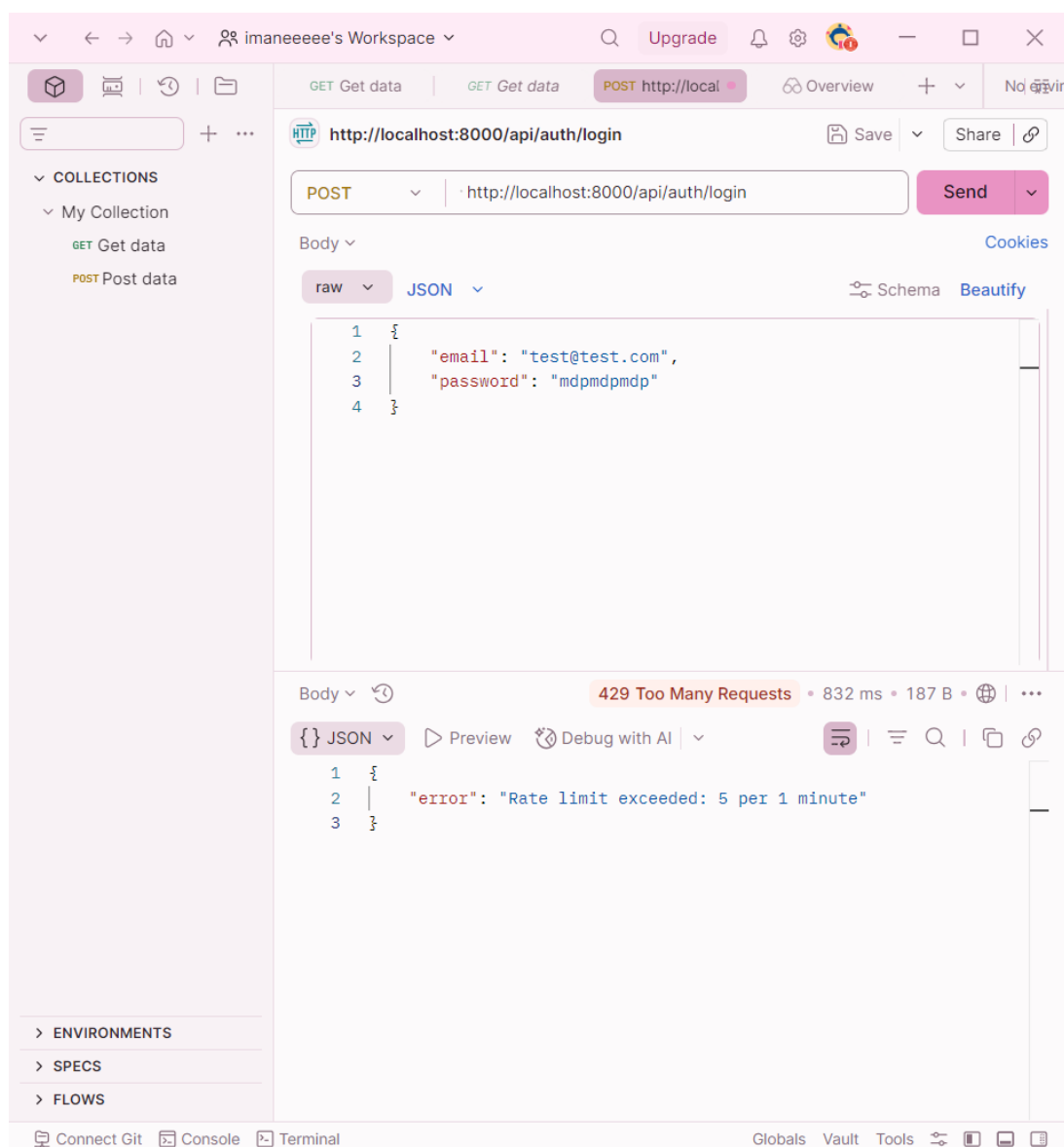


FIGURE 4.7 – T-RATE-01 — 6e requête login → HTTP 429 "Rate limit exceeded : 5 per 1 minute"

4.6 T-RBAC — Contrôle d'Accès

🔧 Test de Vérification

Test 1 : POST /api/releases/ avec token opérateur → attendu 403

Test 2 : GET /api/alerts/critical avec token opérateur → attendu 403

Test 3 : GET /api/alerts/critical (2e vérification avec token directeur) → attendu 200

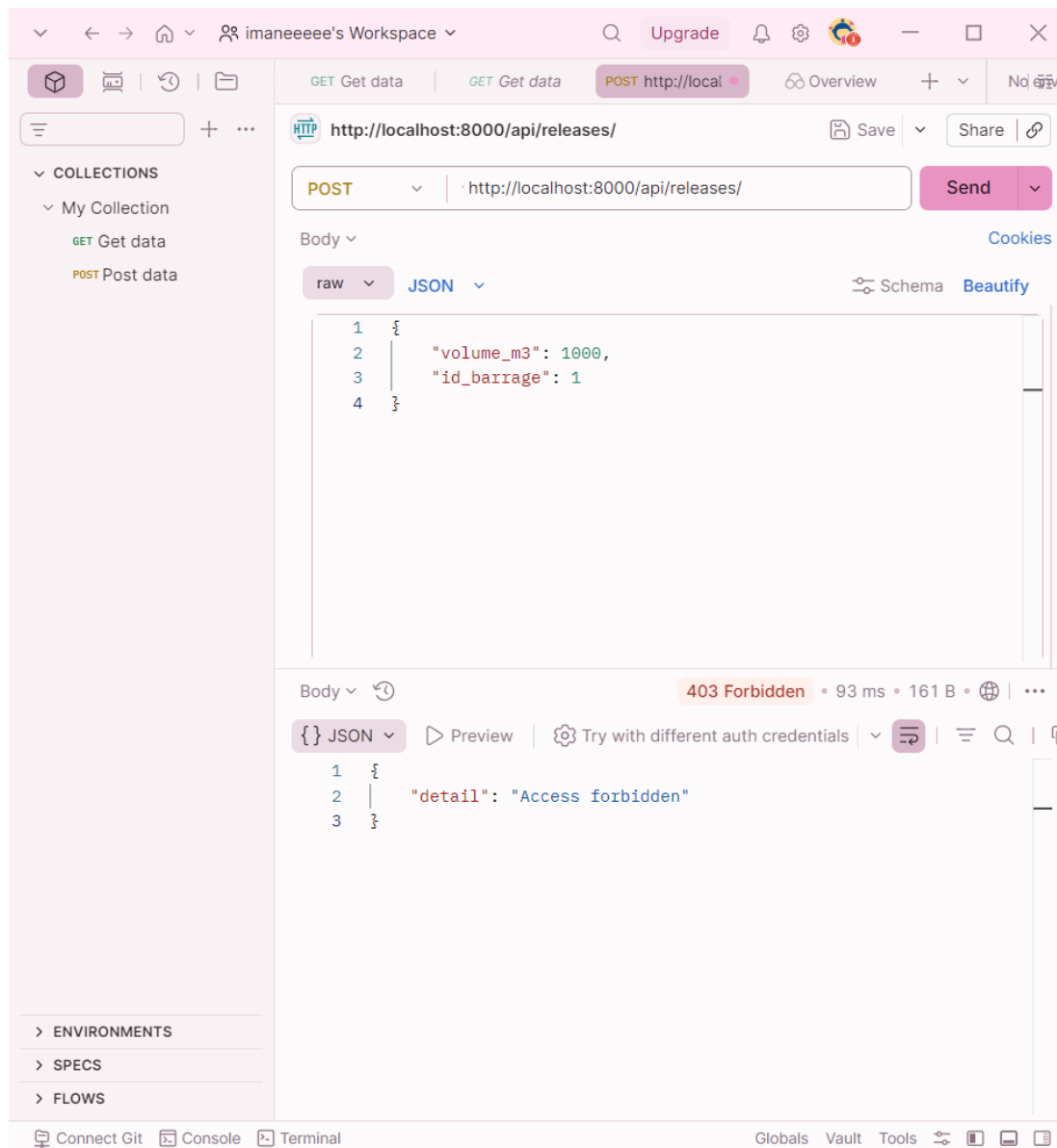


FIGURE 4.8 – T-RBAC-01 — POST /releases/ avec compte opérateur → HTTP 403

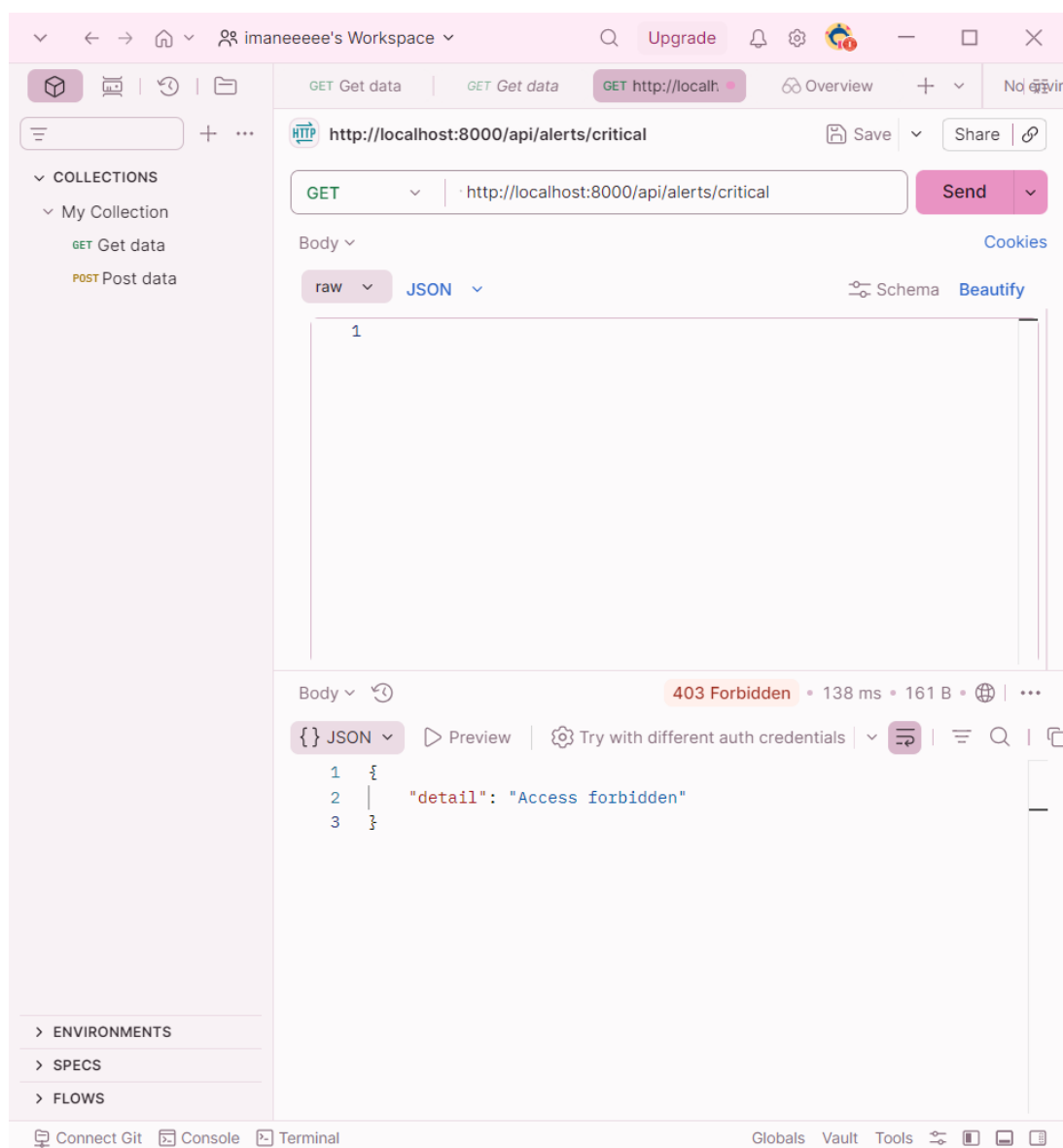


FIGURE 4.9 – T-RBAC-02 — GET /alerts/critical → HTTP 403

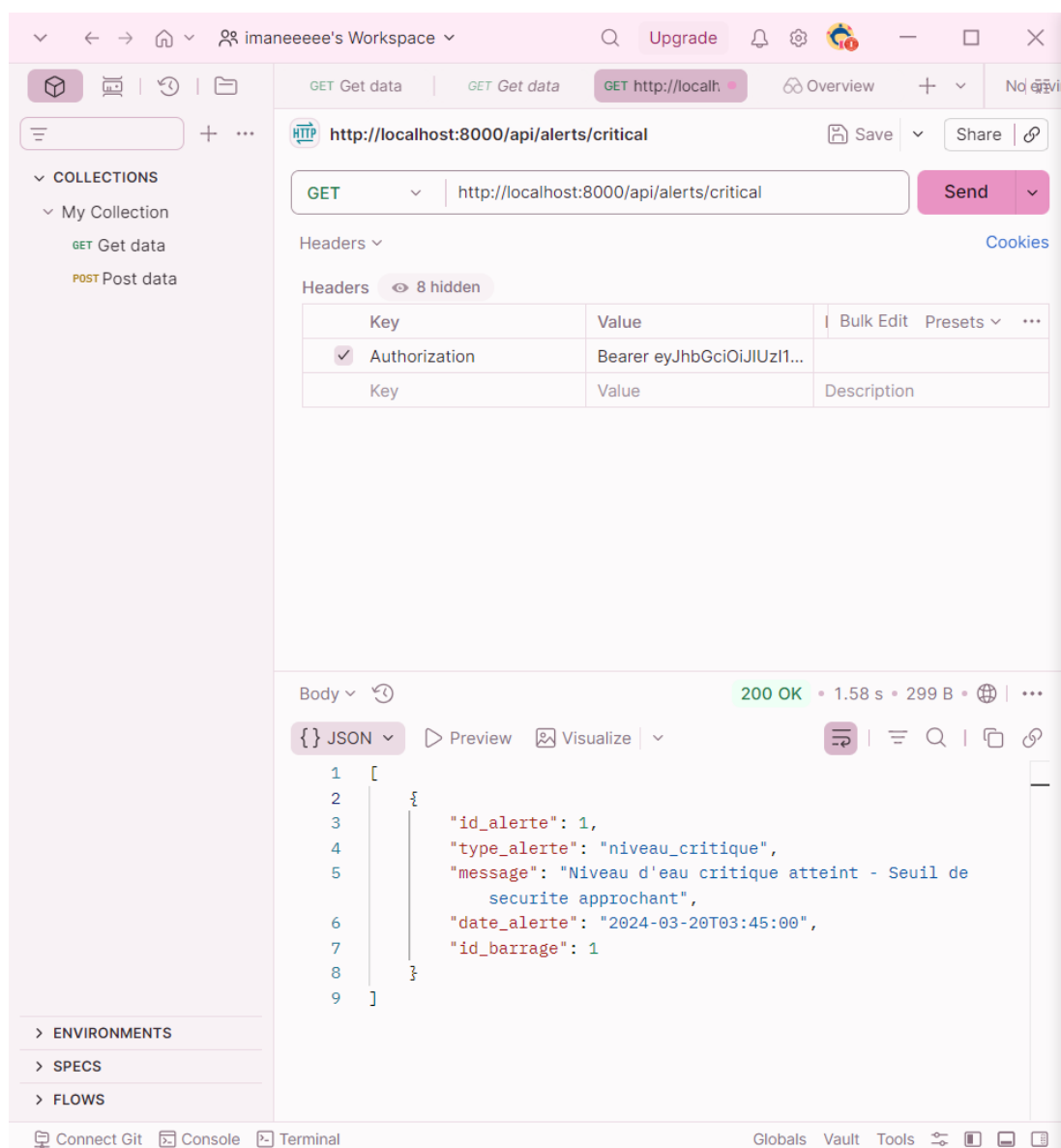


FIGURE 4.10 – T-RBAC-03 — GET /alerts/critical (2e vérification) → 200 OK

4.7 T-HASH-01 — Hashage des Mots de Passe

🔍 Test de Vérification

Test : `SELECT email, password FROM utilisateur`

Résultat attendu : tous les mots de passe commencent par `$2b$12$...`

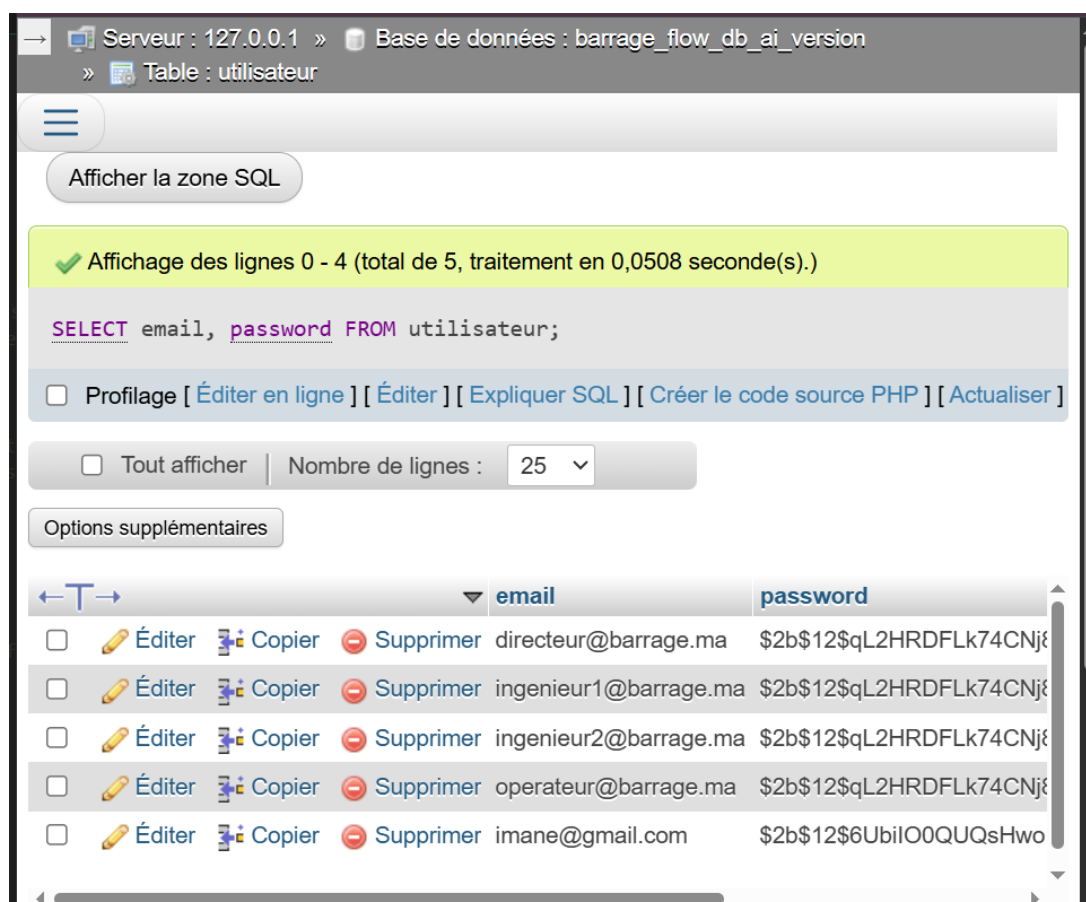


FIGURE 4.11 – T-HASH-01 — Mots de passe hashés bcrypt \$2b\$12\$ en base de données

4.8 T-MYSQL — Permissions MySQL & Port

🔧 Test de Vérification

Test 1 : SHOW GRANTS FOR 'barrage_app'@'%' → uniquement SELECT/INSERT/UPDATE/DELETE/EXECUTE

Test 2 : DROP TABLE utilisateur → attendu ERROR 1142

Test 3 : mysql -h 127.0.0.1 -P 3306 -u root -p → attendu ERROR 2002



FIGURE 4.12 – T-MYSQL-01 — SHOW GRANTS → SELECT/INSERT/UPDATE/-DELETE/EXECUTE uniquement (pas de GRANT ALL)

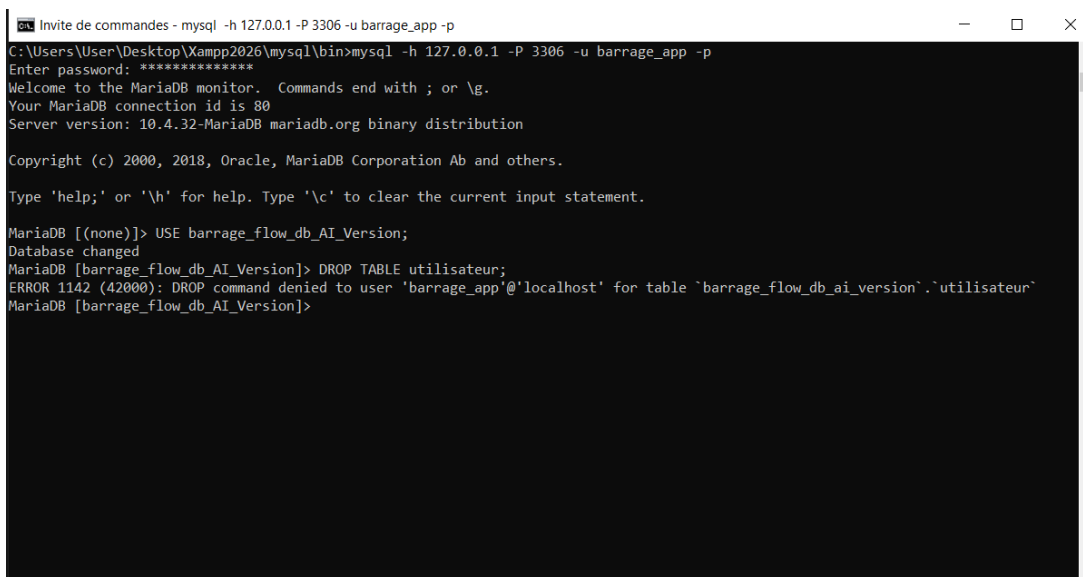
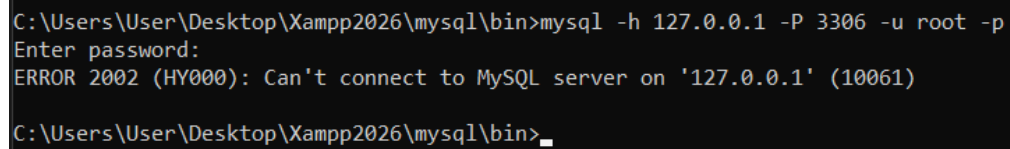


FIGURE 4.13 – T-MYSQL-02 — DROP TABLE utilisateur → ERROR 1142 refusé



```
C:\Users\User\Desktop\Xampp2026\mysql\bin>mysql -h 127.0.0.1 -P 3306 -u root -p
Enter password:
ERROR 2002 (HY000): Can't connect to MySQL server on '127.0.0.1' (10061)
C:\Users\User\Desktop\Xampp2026\mysql\bin>
```

FIGURE 4.14 – T-DOCK-01 — Connexion externe port 3306 → ERROR 2002 (port inaccessible)

5. Bilan Final

5.1 Tableau Récapitulatif des Tests

ID	Défense	Test	HTTP	Résultat
T-SQL-01	Injection SQL	f-string via id_lacher	422	Bloqué
T-PYD-01	Pydantic	volume_m3 = -100	422	Rejeté
T-PYD-02	Pydantic	id_barrage = -5	422	Rejeté
T-PYD-03	Pydantic	password = "ab"	422	Rejeté
T-CORS-01	CORS	Origine malveillante	405	Bloqué
T-RATE-01	Rate Limiting	6 logins / 1 minute	429	Bloqué
T-RBAC-01	RBAC	POST /releases/ (ingénieur)	403	Interdit
T-RBAC-02	RBAC	GET /alerts/critical	403	Interdit
T-RBAC-03	RBAC	GET /alerts/critical (2e)	403	Interdit
T-HASH-01	Hashage bcrypt	SELECT password	—	\$2b\$12\$...
T-MYSQL-01	MySQL privileges	SHOW GRANTS	—	Minimal
T-MYSQL-02	MySQL privileges	DROP TABLE	—	Refusé
T-DOCK-01	Port Docker	mysql -P 3306	—	Refusé

TABLE 5.1 – Tableau récapitulatif — 13/13 tests réussis

5.2 Statistiques

Métrique	Valeur
Total tests réalisés	13
Tests réussis	13 / 13
Taux de réussite	100 %
Vulnérabilités critiques corrigées	6
Vulnérabilités élevées corrigées	5
Vulnérabilités identifiées (futures)	2
Couverture OWASP Top 10	8/10 applicables

TABLE 5.2 – Métriques finales — Blue Team

5.3 Conclusion et Remerciements

Conclusion

La Blue Team (HRIMICH Reda et IGHRANE Imane) a réalisé un audit complet de sécurité du système **Barrage-Flow-Manager** selon le référentiel **OWASP Top 10 2021**. Sur les 8 catégories applicables, **7 ont été entièrement corrigées** avec preuves de tests. Deux vulnérabilités ont été identifiées et documentées comme améliorations futures.

L'application est désormais protégée contre : injection SQL, brute-force, escalade de privilèges, exposition de données sensibles et compromission de la base de données.

Nous remercions notre professeur **Pr. S. EL-ATEIF** pour sa confiance et l'opportunité de contribuer à la sécurité de ce projet d'infrastructure critique, ainsi que l'ensemble de l'équipe pour leur travail collaboratif.

Blue Team — HRIMICH Reda & IGHRANE Imane

Barrage-Flow-Manager Security Hardening — Avril 2026